



UNIVERSIDAD DE CIENCIAS EMPRESARIALES Y SOCIALES

Carrera: Tecnicatura Universitaria en Programación

Asignatura: Programación II

Nombre del/a docente: Mario Daniel Detke

Nombre del/a estudiante: Gonzalo Ezequiel Beloqui

Fecha de entrega: 2026/04/21

Actividad Número 2 - Técnicas de entrada/salida y manejo de archivos

Diseño de archivos de texto para persistencia de clases

Para aplicar el concepto de persistencia se diseñaron dos archivos independientes que permiten guardar y recuperar datos correspondientes a dos clases principales del sistema: Paciente y Profesional. El primer archivo se denomina `pacientes.txt` y almacena la información de los objetos de la clase Paciente. Cada línea del archivo representa un paciente completo, utilizando una estructura de campos separados por punto y coma. Esta organización permite que los datos puedan guardarse como texto plano y, posteriormente, recuperarse para reconstruir los objetos dentro del sistema.

Para administrar este archivo se implementó la clase `GestorPacientesTexto`, ubicada en el paquete controlador. Esta clase contiene los métodos necesarios para guardar una lista de pacientes, agregar un nuevo paciente al archivo y leer los registros existentes para reconstruir objetos Paciente.

- [GestorPacientesTexto](#)

Las operaciones principales son:

```
- guardarPacientes(List<Paciente> pacientes)
- agregarPaciente(Paciente paciente)
- leerPacientes()
```

El guardado completo sobrescribe el archivo, mientras que la operación de agregado utiliza escritura en modo append para incorporar nuevos registros sin eliminar los existentes.

La estructura definida para cada registro de paciente es la siguiente:

```
idPaciente;nombre;apellido;dni;telefono;numeroHistoriaClinica;obraSocial;email
```

El segundo archivo se denomina profesionales.txt y almacena la información de los objetos de la clase Profesional. Al igual que en el caso anterior, cada línea representa un profesional completo, con sus atributos separados por punto y coma.

- [GestorProfesionalesTexto](#)

La estructura definida para cada registro de profesional es la siguiente:

```
idProfesional;nombre;apellido;dni;telefono;matricula;especialidad;emailInstitucion  
al
```

Para administrar este archivo se implementó la clase GestorProfesionalesTexto, también ubicada en el paquete controlador. Los principales métodos son:

```
- guardarProfesionales(List<Profesional> profesionales)  
- agregarProfesional(Profesional profesional)  
- leerProfesionales()
```

En ambos casos, las clases Paciente y Profesional sobrescriben el método toString() para generar una representación en cadena de caracteres compatible con la estructura del archivo. Además, se agregaron métodos estáticos fromString(String linea) que permiten interpretar cada línea leída y convertirla nuevamente en un objeto del sistema. Las operaciones de lectura y escritura utilizan BufferedReader, BufferedWriter, FileReader y FileWriter, aplicando manejo de excepciones mediante try-with-resources, según lo observado en el contenido de la unidad de estudio.

Diseño de estructura XML para persistencia de una clase

La persistencia bajo formato XML se eligió para guardar objetos de la clase Agenda. El archivo generado se denomina agenda.xml y utiliza una etiqueta raíz, que contiene uno o más elementos, donde cada agenda posee etiquetas internas que representan sus atributos principales: 'idAgenda', 'diaSemana', 'horaInicio', 'horaFin' y 'estado', además de la relación con 'idProfesional'.

Por ejemplo, la secuencia concreta de los objetos testeados es la siguiente:

```
<agendas>  
  <agenda>  
    <idAgenda>1</idAgenda>  
    <idProfesional>1</idProfesional>  
    <diaSemana>Lunes</diaSemana>  
    <horaInicio>08:00</horaInicio>  
    <horaFin>12:00</horaFin>
```

```
<estado>Activa</estado>
</agenda>
</agendas>
```

Para implementar esta funcionalidad se creó la clase GestorAgendasXML, ubicada en el paquete controlador, con los siguientes métodos:

```
- guardarAgendas(List<Agenda> agendas)
- leerAgendas()
- leerAgendas(List<Profesional> profesionales)
```

En el diseño utilizan las clases de JAXP y DOM, tales como DocumentBuilderFactory, DocumentBuilder, Document, Element, NodeList, TransformerFactory y Transformer.

- [GestorAgendasXML](#)

Modificación de clases de Actividad 1

I. Agregado de Identificadores autoincrementales

En la versión actual del proyecto se incorporan identificadores numéricos autoincrementales en las clases Paciente, Profesional, Agenda, Turno y Notificacion. En todos los casos, cada clase administra su propio contador estático y asigna automáticamente un valor entero secuencial a cada nuevo objeto creado.

II. Conversión entre objetos y registros de archivo

Las clases 'Paciente' y 'Profesional' fueron ampliadas para soportar serialización textual manual, el método toString() no se usa solo para mostrar información en pantalla, sino también para generar un registro con un formato fijo, donde los atributos se separan mediante ;. A su vez, los métodos fromString(String linea) permiten leer cada línea del archivo, separar sus campos y reconstruir el objeto correspondiente dentro del sistema.

Adicionalmente, para que el guardado en archivos de texto funcione de forma ordenada, la clase Persona (superclase de Paciente y Profesional) incorpora una pequeña validación previa sobre los valores de texto. Mediante el método normalizarTexto(String valor), se eliminan espacios innecesarios (utilizando "trim") y se reemplaza el carácter ";" por ",".

III. Menú interactivo por consola

La clase 'Principal' fue ampliada con un menú de consola que permite ejecutar operaciones del sistema sin modificar el código fuente.

```
--- Sistema de turnos ---
1. Agregar paciente
2. Agregar profesional
3. Agregar agenda
4. Buscar paciente
```

- 5. Buscar profesional
- 6. Buscar agenda por profesional y día
- 0. Salir

Escriba "cancelar" dentro de una opción para volver al menú.